☁️ **Cloud Computing**

📌 Definition:

Cloud computing is the delivery of computing services such as servers, storage, databases, networking, and software over the internet. Instead of buying and maintaining physical hardware, users can rent resources from cloud providers and pay only for what they use. This makes systems flexible, scalable, and cost-effective.

⚠️ Problems with Traditional IT:

- High cost
- Slow scaling
- Maintenance

🚀 Cloud Computing Solution:

Cloud computing solves these issues by providing resources over the internet on demand.

- Resources can be created instantly
- Easy to scale up or down anytime
- No need to manage physical hardware
- Access services globally
- Focus more on development instead of infrastructure

📊 Key Characteristics of Cloud:

- On-demand self-service → users can create resources without manual support
- Broad network access → services available from anywhere via internet
- Resource pooling → shared infrastructure among multiple users
- Rapid elasticity → automatic scaling based on demand
- Measured service → pay only for what you use

🧩 Types of Cloud Computing:

- IaaS → provides infrastructure like virtual machines and storage (e.g., EC2)
- PaaS → provides platform to build and deploy applications
- SaaS → ready-to-use software over internet (e.g., Gmail, Dropbox)

🌍 Deployment Models:

- Public Cloud → services provided over internet (AWS, Azure, Google Cloud)
- Private Cloud → dedicated cloud for one organization
- Hybrid Cloud → combination of public and private cloud

💰 Cloud Pricing Model:

Cloud uses a pay-as-you-go model:

- Compute → pay for server usage time
- Storage → pay for stored data
- Data Transfer → pay mainly for data sent out from cloud

☁️ **AWS Identity and Access Management (IAM)**

📌 Definition:

AWS Identity and Access Management (IAM) is a global service that helps you manage access to AWS. It controls who can access AWS resources and what actions they can perform.

👤 Users & Groups:

IAM manages users (people) and groups (collection of users). The root account is created by default and should not be used regularly. Users can belong to multiple groups, and groups contain only users.

📜 Permissions (Policies):

Permissions are defined using JSON policies and assigned to users or groups. These policies define what actions are allowed or denied. Always follow the least privilege principle, meaning give only the required permissions.

🧩 Policy Structure:

A policy includes version, statement, effect (allow or deny), action (what can be done), resource (on what), and optional conditions.

🔒 Password Policy:

- Minimum length and strong password rules
- Use uppercase, lowercase, numbers, symbols
- Password expiry and no reuse

🔐 Multi-Factor Authentication (MFA):

Multi-Factor Authentication (MFA) adds extra security using password + device (OTP).

- Protects account even if password is stolen

💻 Accessing AWS:

- Management Console (UI access)
- Command Line Interface (CLI)
- Software Development Kit (SDK)

🔑 Access Keys:
- Used for CLI / SDK access
- Access Key ID = username
- Secret Access Key = password
- Never share these keys

⚙️ AWS Command Line Interface (CLI):
Tool to interact with AWS services using commands and automate tasks.
🧑‍💻 AWS Software Development Kit (SDK):
Provides libraries (Java, Python, JavaScript, etc.) to interact with AWS services inside applications.
🔁 IAM Roles:
Used to give permissions to AWS services like EC2 and Lambda without using access keys.
📊 Security Tools:
- Credentials Report → shows user credential status
- Access Advisor → shows permission usage

✅ Best Practices:
- Avoid using root account
- One user per person
- Use groups for permissions
- Enable MFA
- Use roles for services
- Don't share access keys
- Audit permissions regularly

☁️ **Amazon EC2 (Elastic Compute Cloud)**
📌 Definition:
Amazon EC2 (Elastic Compute Cloud) is an Infrastructure as a Service (IaaS) that allows you to rent virtual machines in the cloud. It helps you run applications without managing physical servers.
⚙️ What EC2 Provides:
- Virtual machines (EC2 instances)
- Storage (EBS)
- Load balancing (ELB)
- Auto scaling (ASG)

🧩 EC2 Configuration Options:
- Operating System → Linux, Windows, Mac
- CPU → compute power
- RAM → memory
- Storage → EBS, EFS, Instance Store
- Network → IP, speed
- Security → Security Groups
- User Data → startup script

🚀 EC2 User Data:
EC2 User Data is used to run scripts when an instance starts (only first time).
- Install updates and software
- Download files
- Automate setup

🖥️ EC2 Instance Types:
Different instance types are optimized for different use cases.
- General Purpose → balanced (t2.micro)
- Compute Optimized → high CPU tasks
- Memory Optimized → large data in memory
- Storage Optimized → high storage performance

🔐 Security Groups:
Security Groups act as a firewall for EC2 instances.
- Control inbound and outbound traffic
- Work using IP and port rules
- All inbound blocked by default
- All outbound allowed by default

📡 Key Ports:
- 22 → SSH (Linux login)
- 21 → FTP

- 80 → HTTP
- 443 → HTTPS
- 3389 → RDP (Windows)

🔑 Access EC2:
- SSH → connect to Linux instance
- Putty → for Windows
- EC2 Instance Connect → browser-based access

💰 EC2 Pricing Options:
- On-Demand → pay per use
- Reserved → long-term discount
- Savings Plans → commit usage
- Spot Instances → very cheap, can stop anytime

⚡ Spot Instances:
- Up to 90% cheaper
- Can be terminated anytime
- Good for non-critical workloads

🏢 Dedicated Hosts & Instances:
- Dedicated Hosts → full server control
- Dedicated Instances → dedicated hardware

📦 Capacity Reservations:
- Reserve EC2 capacity in specific AZ
- Pay even if not used

🌐 Public IP Charges:
- Public IPv4 is chargeable (~$0.005/hour)
- Free tier: limited usage for new users

⚙️ Spot Fleet:
- Collection of Spot + On-Demand instances
- Automatically selects cheapest options
- Helps optimize cost and availability

☁️ **Amazon EC2 – Advanced**

📌 Private vs Public IP:
- IPv4 is most commonly used (e.g., 1.160.10.240)
- Public IP → accessible over internet, must be unique
- Private IP → used inside internal network
- Private IP can be same across different networks
- Internet access uses NAT + Internet Gateway

🌐 EC2 IP Behavior:
- EC2 gets both private and public IP by default
- Public IP is used for internet access (SSH, etc.)
- Public IP can change after stop/start

🔗 Elastic IP:
- Fixed public IP address
- Doesn't change after restart
- Can attach to one instance at a time
- Limited (default 5 per account)
- Prefer using DNS or Load Balancer instead

📦 Placement Groups:
Used to control how EC2 instances are placed.
- Cluster → same AZ, low latency, high speed
- Spread → across hardware, high availability
- Partition → multiple groups, big distributed systems

🔌 Elastic Network Interface (ENI):
- Virtual network card for EC2
- Has private/public IP, security groups
- Can be attached/detached from instances
- Used for failover

💤 EC2 Hibernate:
- Saves RAM state to EBS

- Faster restart (no full boot)
- Useful for long-running apps

📌 Key Notes:

- Public IP changes → use Elastic IP if needed
- Security groups control network access
- Placement groups improve performance & availability

☁️ **Amazon EC2 – Storage (EBS, EFS, AMI)**

📌 What is EBS (Elastic Block Store):

Amazon EBS is a network storage (like a virtual hard disk) that you attach to EC2 instances. It stores data even after instance stops or terminates.

🔧 EBS Features:

- Network-based storage (slight latency)
- Can attach/detach to EC2
- Works in same Availability Zone
- Size and performance can be increased
- Billed for allocated storage

📦 EBS Snapshots:

- Backup of EBS volume at a point in time
- Used to move data across AZ or region
- Can restore volume from snapshot

🗑️ Delete on Termination:

- Root volume → deleted by default
- Extra volumes → not deleted
- Can be configured

💿 AMI (Amazon Machine Image):

AMI is a pre-configured template used to launch EC2 instances.

- Includes OS, software, configuration
- Faster instance launch
- Types:
    - AWS AMI (default)
    - Custom AMI
    - Marketplace AMI

⚡ EC2 Instance Store:

- Physical storage attached to EC2
- Very high performance
- Data lost when instance stops
- Used for cache, temp data

📊 EBS Volume Types:

- gp2 / gp3 → general purpose SSD
- io1 / io2 → high performance SSD
- st1 → throughput HDD
- sc1 → low-cost HDD

🔁 EBS Multi-Attach:

- Same volume attached to multiple EC2 (same AZ)
- Used for high availability systems

🔐 EBS Encryption:

- Data encrypted at rest and in transit
- Uses AWS KMS keys
- No performance impact

🌐 Amazon EFS (Elastic File System):

EFS is a shared file system that can be used by multiple EC2 instances.

📁 EFS Features:

- Works across multiple AZ
- Auto scalable
- Pay as you use
- Used for shared data (like websites)

📊 EFS Storage Types:

- Standard → frequently used data
- Infrequent Access → lower cost

- Archive → rarely used data

⚖️ EBS vs EFS:

- EBS → single instance (mostly), high performance
- EFS → multiple instances, shared storage
- EFS is more expensive but scalable

☁️ **High Availability & Scalability**

📌 Definition:
Scalability means a system can handle more load by adding or upgrading resources. High Availability means the system keeps running even if some parts fail.

📈 Types of Scalability:

🔹 Vertical Scalability:
Increase power of one server (scale up)
Example → t2.micro to t2.large
Common for databases (RDS, ElastiCache)
Limited by hardware

🔹 Horizontal Scalability:
Add more servers instead of upgrading one
Used in distributed systems
Very common in web applications

🏢 High Availability:
Run application in multiple Availability Zones
If one AZ fails, others continue working
Can be:

- Passive → standby (RDS Multi-AZ)
- Active → multiple running instances

⚙️ EC2 Scaling:
Vertical → increase instance size
Horizontal → Auto Scaling Group + Load Balancer
High Availability → deploy in multiple AZ

🌐 Load Balancer:
Load balancer distributes incoming traffic to multiple EC2 instances

📌 Why use Load Balancer:

- Spread traffic across servers
- Single access point (DNS)
- Handles failures automatically
- Performs health checks
- Supports HTTPS (SSL)

🔍 Health Checks:
Load balancer checks if instance is working
Uses path like /health
If response is not OK → instance is removed from traffic

📊 Types of Load Balancer:
Classic → old generation
Application → HTTP/HTTPS (Layer 7)
Network → TCP/UDP (Layer 4)
Gateway → network security

🔐 SSL / TLS:
Encrypts data between user and server
Managed using AWS Certificate Manager

⚡ Auto Scaling Group (ASG):
Automatically manages EC2 instances based on load

📌 ASG Features:

- Add instances when load increases
- Remove instances when load decreases
- Maintain min, max, desired capacity
- Replace unhealthy instances
- Works with Load Balancer

⚙️ Scaling Policies:
Target Tracking → keep CPU at target level
Step Scaling → add/remove based on rules
Scheduled Scaling → based on time

📊 Metrics:
CPU Utilization
Request count per instance
Network traffic
⏳ Cooldown:
Wait time after scaling (default ~300 sec) to stabilize system

---

☁️ **Amazon RDS, Aurora & ElastiCache**
📌 Amazon RDS (Relational Database Service):
Amazon RDS is a managed database service that allows you to run SQL databases in the cloud. AWS handles setup, backups, patching, and maintenance.
📌 Supported Databases:

- PostgreSQL
- MySQL
- MariaDB
- Oracle
- SQL Server
- DB2
- Aurora

⚙️ RDS Features:

- Automated backups and restore
- Monitoring and maintenance by AWS
- Read Replicas for read scaling
- Multi-AZ for high availability
- Easy scaling (mainly vertical)

📦 Storage Auto Scaling:

- Automatically increases storage when low
- No manual work required

📊 Read Replicas:

- Used for handling read traffic
- Up to 15 replicas
- Async replication (small delay possible)
- Only for read operations (SELECT)

🏢 Multi-AZ (High Availability):

- Used for failover, not scaling
- Sync data to standby DB
- Automatic failover
- Same DNS (no app change needed)

💿 Amazon Aurora:
Aurora is a high-performance AWS database (compatible with MySQL/PostgreSQL).
⚡ Aurora Features:

- Faster than standard RDS
- Auto-scaling storage (up to 128 TB)
- Up to 15 read replicas
- High availability by default
- Fast failover

⚡ Aurora Serverless:

- Auto start/stop and scale
- Good for unpredictable workloads
- Pay per use

🌍 Global Aurora:

- Multi-region setup
- Very low replication delay
- Used for disaster recovery

🔐 RDS & Aurora Security:

- Encryption using KMS
- Secure transfer using SSL/TLS
- IAM authentication
- Security Groups control access

🔌 RDS Proxy:

- Manages DB connections
- Improves performance
- Reduces DB load

⚡ Amazon ElastiCache:
ElastiCache is a managed in-memory database (Redis / Memcached).
📌 Why use ElastiCache:
- Very fast performance
- Reduces DB load
- Improves app speed

🧩 Use Cases:
- Database caching
- Session storage
- Real-time data

🔁 Redis vs Memcached:
- Redis → supports replication, backup, advanced data
- Memcached → simple, fast, no persistence

📌 Cache Patterns:
- Lazy Loading → load data when needed
- Write Through → update cache with DB
- Session Store → store user sessions

☁️ **Amazon Route 53**
📌 Definition:
Amazon Route 53 is a highly available, scalable and fully managed DNS service. It converts domain names into IP addresses and also works as a domain registrar.
🌐 What is DNS:
DNS (Domain Name System) converts domain name → IP address
Example → google.com → 172.x.x.x
DNS is backbone of internet and uses hierarchical structure
📌 DNS Terminologies:
- Domain Registrar → where you buy domain (Route 53, GoDaddy)
- DNS Records → A, AAAA, CNAME, NS
- Name Server → resolves DNS queries
- TLD → .com, .in, .org
- SLD → example.com
- FQDN → full domain name

⚙️ Route 53 Features:
- Highly available (100% SLA)
- Fully managed and scalable
- Authoritative DNS (you control records)
- Health checks supported
- Domain registration support

📄 Route 53 Records:
Each record defines routing behavior
- Name → domain/subdomain
- Type → A, AAAA, CNAME, NS
- Value → IP or domain
- Routing Policy → how response is given
- TTL → cache duration

📌 Record Types:
- A → domain to IPv4
- AAAA → domain to IPv6
- CNAME → domain to another domain (not root)
- NS → name servers

🧩 Hosted Zones:
Container for DNS records
- Public Hosted Zone → internet traffic
- Private Hosted Zone → inside VPC
- Cost → ~$0.50/month per zone

⏳ TTL (Time To Live):

- High TTL → less cost, slower updates
- Low TTL → faster updates, more cost
- Mandatory for records (except Alias)

🔗 CNAME vs Alias:

- CNAME → maps to another domain (only subdomain)
- Alias → maps to AWS resources
- Alias works for root domain also
- Alias is free and supports health checks

📌 Alias Records:

- Used for ELB, CloudFront, S3, API Gateway, etc.
- Auto handles IP changes
- No TTL required
- Cannot be used for EC2 public DNS

🎯 Alias Targets:

- ELB
- CloudFront
- API Gateway
- S3 Website
- Elastic Beanstalk
- Global Accelerator
- VPC Endpoints

🔁 Routing Policies:
Define how DNS responds (not actual traffic routing)

- Simple → single or multiple values
- Weighted → split traffic %
- Latency → lowest latency region
- Failover → primary + backup
- Geolocation → based on user location
- Geoproximity → based on distance + bias
- Multi-Value → multiple healthy IPs
- IP-based → based on client IP

💗 Health Checks:

- Monitor endpoint (HTTP/HTTPS/TCP)
- Works only for public resources
- Uses multiple global checkers
- Based on response (2xx/3xx = healthy)
- Can integrate with CloudWatch
- Supports calculated health checks

📌 Failover Routing:

- Primary + Secondary setup
- Automatic switch on failure
- Health check required

📌 Key Notes:

- DNS only answers queries, does not route traffic
- Alias preferred for AWS resources
- Registrar and DNS service can be different

☁️ **Classic Solutions Architecture**
📌 **Overview**
Shows how AWS services work together to build scalable and highly available applications.

☁️ **Stateless Web App**
📌 **Concept**
Simple application without database or user session.
⚙️ **Architecture Flow**
User → Route53 → Load Balancer → EC2

📈 **Scaling Progression**

- Single EC2 with Elastic IP

- Vertical Scaling → increase instance size
- Horizontal Scaling → multiple EC2 instances
- Load Balancer → distribute traffic
- Auto Scaling Group → automatic scaling
- Multi-AZ → high availability

📌 **Key Learnings**

- Difference between Elastic IP, Route53, and Load Balancer
- Vertical vs Horizontal Scaling
- Multi-AZ improves availability
- Security Groups control access

☁️ **Stateful Web App**

📌 **Concept**

Application with users, sessions, and database.

⚙️ **Challenges**

- User session management
- Data storage
- High traffic handling

🔧 **Solutions**

Session Management

- Sticky Sessions
- Cookies
- Backend Session Storage

Best Practice

Use ElastiCache or DynamoDB for session storage.

Database Layer

- RDS for relational data
- Read Replicas for read scaling
- Multi-AZ for high availability

Performance

Use ElastiCache for caching.

Security

Use Security Groups with minimum required access.

☁️ **WordPress Architecture**

📌 **Architecture**

```
User → ALB → EC2 (ASG) → RDS/Aurora
            ↓
            EFS
```

📌 **Storage**

- EBS → single instance storage
- EFS → shared storage across instances

📌 **Database**

Aurora provides better scalability and availability.

☁️ **Fast Application Deployment**

📌 **Problem**

Launching infrastructure manually takes time.

🔧 **Solutions**

- Golden AMI → preconfigured server image
- User Data → startup automation
- RDS Snapshot → database restore
- EBS Snapshot → volume restore

☁️ **3-Tier Architecture**

📌 **Layers**

Presentation Layer

- Route53
- Load Balancer

Application Layer

- EC2
- Auto Scaling Group

Data Layer

- RDS
- ElastiCache

📌 **Features**

- Multi-AZ deployment
- High availability
- Scalability
- Security separation

☁️ **Elastic Beanstalk**

📌 **Definition**

Managed service for deploying applications quickly.

⚙️ **Features**

- Automatically manages EC2, ELB, ASG, and RDS
- Supports auto scaling
- Developer focuses on application code

📌 **Components**

- Application → project
- Version → application version
- Environment → deployed infrastructure

📌 **Environment Types**

- Web Tier → handles HTTP requests
- Worker Tier → background processing using SQS

📌 **Deployment Modes**

- Single Instance → development/testing
- Load Balanced → production

📌 **Supported Platforms**

Java, Node.js, Python, PHP, .NET, Docker, and more.

☁️ **Amazon S3 (Simple Storage Service)**

📌 Definition:

Amazon S3 is a highly scalable object storage service used to store and retrieve any amount of data from anywhere. It is one of the core AWS services and supports unlimited storage.

📌 Use Cases:

- Backup and storage
- Disaster recovery
- Archive data
- Static website hosting
- Media hosting
- Big data & analytics
- Software delivery

📦 Buckets:

Buckets are containers to store files (objects)

- Must be globally unique
- Created in a specific region
- Naming rules:
  - 3–63 characters
  - Lowercase letters and numbers only
  - No underscore or uppercase
  - Not an IP format

📄 Objects:

Objects are files stored inside buckets

- Each object has a Key (full path)
  Example → s3://bucket/folder/file.txt
- Max size → 5 TB

- 5GB → use multipart upload
- Includes:
  - Metadata (key-value)
  - Tags (up to 10)
  - Version ID (if enabled)
    ⚠️ Note:
    S3 has no real folders, only key paths

🔐 Security:
Access control in S3

- IAM Policies → user-based access
- Bucket Policies → resource-based access
- ACLs → object/bucket level (less used)

📌 Access Rule:
User can access if:

- IAM allows OR
- Resource policy allows
- AND no explicit DENY

📌 Bucket Policy Use Cases:

- Make bucket public
- Force encryption
- Cross-account access

🚫 Block Public Access:

- Prevent accidental public exposure
- Can be set at bucket or account level

🌐 Static Website Hosting:
S3 can host static websites

- URL format:
  http://bucket-name.s3-website-region.amazonaws.com
- Needs public read access
- 403 error → check bucket policy

📌 Versioning:
Stores multiple versions of same file

- Enabled at bucket level
- Protects from accidental delete
- Helps rollback
  ⚠️ Notes:
- Old files get version “null”
- Disabling versioning doesn’t delete old versions

🔁 Replication (CRR & SRR):

- CRR → Cross Region
- SRR → Same Region
- Requires versioning enabled
- Async replication
- Works across accounts

📌 Use Cases:

- Backup & compliance
- Disaster recovery
- Data sharing
  ⚠️ Notes:
- Only new objects replicated
- Delete with version ID not replicated
- No chaining replication

📈 Durability vs Availability:

- Durability → 99.999999999% (11 9’s)
- Availability → depends on storage class
  Example:
- Standard → 99.99%
- IA → 99.9%

📌 Key Points:

- S3 is object storage (not block storage)

- Unlimited storage capacity
- Highly durable across multiple AZ
- Use correct storage class to save cost

☁️ **Amazon S3 – Advanced**
📌 Moving Between Storage Classes:
You can move objects between storage classes based on usage
- Frequently used → keep in Standard
- Less used → move to Standard IA
- Archive data → move to Glacier / Deep Archive
- Can automate using Lifecycle Rules

📌 Lifecycle Rules:
Automate data management
- Transition → move objects to another storage class
- Expiration → delete objects after some time

📌 Examples:
- Move to IA after 60 days
- Move to Glacier after 6 months
- Delete logs after 365 days
- Delete old versions (if versioning enabled)
- Delete incomplete uploads

📌 Filters:
- Based on prefix (folder path)
- Based on object tags

📊 S3 Analytics:
Helps decide best storage class
- Suggests moving Standard → IA
- Updated daily
- Takes 24–48 hours to start
- Does not support Glacier / One Zone IA

💰 Requester Pays:
- Normally owner pays cost
- In this → requester pays download cost
- Useful for sharing large data
- Requester must be authenticated

🔔 Event Notifications:
Trigger actions on S3 events
- Events → ObjectCreated, ObjectRemoved, etc.
- Can filter by file type (e.g., .jpg)
- Targets → Lambda, SQS, SNS
- Used for automation (e.g., thumbnail generation)

📌 EventBridge Integration:
- Advanced filtering (JSON rules)
- Multiple targets (Step Functions, Kinesis)
- Supports event replay and archive

⚡ Performance:
- Auto scales with high request rate
- ~3500 write / 5500 read requests per second per prefix
- No limit on number of prefixes

🚀 Performance Optimization:
- Multipart Upload → for large files (>100MB, required >5GB)
- Transfer Acceleration → faster uploads via edge locations
- Byte-Range Fetch → download parts of file in parallel

☁️ **Amazon S3 – Security**
📌 Object Encryption:
S3 uses encryption to protect stored data
- SSE-S3 → AWS manages keys (default, simple)
- SSE-KMS → uses AWS KMS (more control + logging)
- SSE-C → you provide your own key

- Client-side → encrypt before uploading

📌 Key Understanding:

- SSE-S3 → easiest, no setup
- SSE-KMS → better security + audit but has API limits
- SSE-C → full control but more responsibility
- Client-side → full control, most secure

🔐 Encryption in Transit:
Data should always move securely

- HTTP → not secure
- HTTPS → secure (recommended)
- Required for SSE-C

📌 Force Encryption:
You can force security using bucket policy

- Allow only HTTPS requests
- Deny uploads without encryption

📌 Default Encryption vs Policy:

- Default encryption → auto applied
- Bucket policy → strict rules (more control)
- Policy is checked before default encryption

🌐 CORS (Cross-Origin Resource Sharing):
Used when frontend and S3 are on different domains

- Browser blocks request by default
- Need to allow using CORS settings
- Can allow specific domain or all (*)

🔑 MFA Delete:
Extra protection for delete operations

- Required to delete object versions
- Required to disable versioning
- Versioning must be enabled
- Only root user can enable

📊 S3 Access Logs:
Tracks all access to bucket

- Logs include allowed + denied requests
- Stored in another bucket
- Used for monitoring and audit

⚠️ Important:

- Never store logs in same bucket (creates loop)

🔗 Pre-Signed URL:
Temporary access to private objects

- Used for secure sharing
- Works for upload and download
- Access expires after time

🧊 S3 Object Lock:
Prevents deletion of data (WORM model)

- Compliance → no one can delete (even root)
- Governance → some users can override

📌 Features:

- Retention period → fixed protection time
- Legal hold → protect without time limit

🧊 Glacier Vault Lock:

- Used for compliance
- Policy cannot be changed once locked

📌 Access Points:
Simplifies access management

- Each access point has its own policy
- Useful for large teams

📌 VPC Access Points:

- Access only from inside VPC
- Requires VPC Endpoint

⚡ S3 Object Lambda:
Modify data before sending to user

- Hide sensitive data
- Convert format
- Resize images

☁️ **Amazon CloudFront**
📌 Definition:
CloudFront is a CDN (Content Delivery Network) that delivers content faster by caching data at edge locations close to users
📌 Key Benefits:

- Faster performance (low latency)
- Content cached near users
- Better user experience
- DDoS protection (AWS Shield + WAF)
- Global edge locations

📌 Origins (Source of Data):
CloudFront fetches data from origin

- S3 Bucket → for static files
- ALB (Application Load Balancer)
- EC2 Instance
- S3 Static Website
- Any HTTP backend

📌 Important:

- Use OAC (Origin Access Control) for secure S3 access
- Replaces older OAI

📌 How it Works:

- User requests data
- CloudFront checks cache (edge location)
- If present → return immediately
- If not → fetch from origin and cache it

📌 CloudFront vs S3 Replication:

- CloudFront → caching (temporary, fast global access)
- S3 CRR → permanent copy in another region

👉 Use:

- CloudFront → fast delivery
- S3 Replication → data backup / availability

📌 Security (EC2 / ALB):

- Allow only CloudFront IPs in security group
- Prevent direct public access

📌 Geo Restriction:
Control access by country

- Allowlist → allow specific countries
- Blocklist → block specific countries

📌 Pricing:

- Cost depends on data transfer location
- Far regions → higher cost

📌 Price Classes:

- Price Class All → best performance (all regions)
- Price Class 200 → most regions
- Price Class 100 → cheapest (limited regions)

📌 Cache Invalidation:
Used when content is updated

- CloudFront caches data for TTL
- Use invalidation to refresh immediately
- Can clear all (*) or specific path

☁️ **AWS Global Accelerator**
📌 Definition:
Global Accelerator improves performance by routing traffic through AWS private network instead of public internet

📌 Key Concept:
- Uses Anycast IP (same IP worldwide)
- Routes user to nearest edge location
- Then traffic goes through AWS network

📌 Benefits:
- Faster performance
- Lower latency
- Better failover (less than 1 minute)
- Static IP (no DNS change issues)
- Health checks for applications

📌 Works With:
- EC2
- ALB / NLB
- Elastic IP

📌 Security:
- Only 2 static IPs to whitelist
- DDoS protection (AWS Shield)

📌 CloudFront vs Global Accelerator:
CloudFront:
- Works on HTTP/HTTPS
- Caches content at edge
- Best for static + web content

Global Accelerator:
- Works on TCP/UDP
- No caching
- Routes traffic faster via AWS network
- Best for real-time apps, gaming, APIs

👉 Simple Difference:
- CloudFront → **content delivery (cache)**
- Global Accelerator → **network routing (speed + path)**

☁️ **AWS Storage Extras**
📌 AWS Snow Family:
Highly-secure, portable devices used to collect, process and transfer data into and out of AWS
👉 Shows network transfer is very slow for large data

☁️ **Amazon FSx (File Systems)**
📌 Definition:
Fully managed high-performance file systems on AWS

☁️ **Hybrid Storage**
📌 Concept:
Part infrastructure on cloud and part on-prem
📌 Why Needed:
- Long migration time
- Security requirements
- Compliance requirements
- Business strategy

📌 Problem:
S3 cannot be directly used as file system on-prem
📌 Solution:
Use AWS Storage Gateway

☁️ **AWS Storage Gateway**
📌 Definition:
Bridge between on-prem data and AWS cloud
📌 Use Cases:
- Disaster recovery
- Backup & restore

☁️ **AWS Transfer Family**
📌 Definition:
Managed service for file transfer into and out of AWS
📌 Protocols:
- FTP
- FTPS
- SFTP

📌 Features:
- Works with S3 / EFS
- Scalable and highly available
- Multi-AZ
- Supports AD, LDAP, IAM
- Pay per endpoint + data transfer

☁️ **AWS DataSync**
📌 Definition:
Service to move large data between systems
📌 Supports:
- On-prem → AWS (needs agent)
- AWS → AWS (no agent)

📌 Features:
- Supports NFS, SMB, HDFS
- Up to 10 Gbps speed
- Scheduled transfers
- Keeps metadata and permissions
- Bandwidth control available

📌 Targets:
- S3 (all classes)
- EFS
- FSx

☁️ **Storage Comparison**
📌 Object Storage:
- S3 → object storage
- Glacier → archive

📌 Block Storage:
- EBS → network storage for EC2

📌 Instance Storage:
- Physical storage on EC2
- Very fast
- Temporary (data lost on stop)

📌 File Storage:
- EFS → Linux file system
- FSx Windows → Windows file system
- FSx Lustre → HPC workloads
- FSx ONTAP / OpenZFS → enterprise

📌 Hybrid & Transfer:
- Storage Gateway → hybrid
- DataSync → data transfer
- Snow Family → physical transfer

📌 Extra:
- Database → structured data with queries

☁️ **AWS Integration & Messaging**
📌 Communication Types:
When multiple applications communicate, there are two patterns
- Synchronous → direct communication between applications without any queue
- Asynchronous → communication happens through a queue which decouples systems

📌 Problem with Synchronous:
- Not good for handling traffic spikes and high load
- Example → system designed for 10 req/sec suddenly gets 1000 req/sec

📌 Solution (Decoupling Services):
- SQS → used for queue-based communication between services
- SNS → used to send messages to multiple subscribers at once
- Kinesis → used for real-time streaming and processing

☁️ **Amazon SQS (Simple Queue Service)**

📌 Definition:
Amazon SQS is a fully managed message queue service used to decouple applications and allow them to scale independently

📌 What is a Queue:
- Producers send messages to queue and consumers read them later
- Queue acts as a buffer between services

📌 Standard Queue:
- Supports unlimited throughput and unlimited number of messages
- Message retention ranges from 4 days to 14 days
- Delivers messages at least once so duplicates can occur
- Does not guarantee strict ordering of messages

📌 Message Flow:
- Producer sends message which is stored in queue
- Consumer polls messages (up to 10 at a time)
- After processing, message must be deleted

📌 Scaling:
- Multiple consumers can process messages in parallel
- Auto scaling can be based on queue length

📌 Visibility Timeout:
- After reading, message becomes temporarily invisible
- If not processed in time, it becomes visible again

👉 Important:
- Low timeout → same message processed multiple times
- High timeout → delay in retry if failure happens

📌 Long Polling:
- Waits for messages up to 20 seconds before returning
- Reduces empty responses and API cost

📌 FIFO Queue:
- Ensures strict ordering of messages
- Prevents duplicate messages
- Throughput is limited compared to standard queue

📌 Security:
- HTTPS ensures secure data transfer
- KMS provides encryption at rest
- IAM and queue policies control access

☁️ **Amazon SNS (Simple Notification Service)**

📌 Definition:
Amazon SNS is a fully managed publish/subscribe messaging service used to send messages from one source to multiple subscribers

📌 How it Works:
- Producer publishes message to SNS topic
- Topic delivers message to all subscribed endpoints

📌 Subscribers:
- SQS queues receive messages for processing
- Lambda functions trigger execution
- Email and SMS deliver notifications
- HTTP endpoints receive webhooks

📌 Key Features:
- Push-based delivery (no polling needed)
- Supports millions of subscribers

- Allows message filtering per subscriber

📌 Fan-Out Pattern:
- One message is sent to multiple SQS queues
- Each system processes message independently

📌 FIFO Topic:
- Maintains order of messages
- Supports deduplication
- Works with FIFO SQS queues

📌 Security:
- HTTPS ensures secure delivery
- KMS encrypts messages
- IAM and topic policies control access

☁️ **Amazon Kinesis**

📌 Definition:
Amazon Kinesis is a service that makes it easy to collect, process, and analyze real-time streaming data

📌 Use Cases:
- Application logs and metrics processing
- Clickstream data from websites
- IoT device data streaming

📌 Types:
- Data Streams → real-time ingestion and processing of data
- Firehose → loads streaming data into storage services
- Analytics → processes data using SQL
- Video Streams → handles video data

📌 Data Streams:
- Stores data for 1 to 365 days for replay
- Data is immutable and cannot be modified

📌 Shards:
- Each shard supports 1 MB/sec write and 2 MB/sec read
- More shards = more throughput

📌 Partition Key:
- Ensures ordering of records with same key
- Same key always goes to same shard

📌 Capacity Modes:
- Provisioned → manually define shards
- On-demand → automatically scales

📌 Security:
- IAM controls access
- HTTPS secures data in transit
- KMS encrypts data
- VPC endpoints for private access

☁️ **Kinesis Data Firehose**

📌 Definition:
Kinesis Data Firehose is a fully managed service used to load streaming data into AWS data stores

📌 Features:
- Automatically scales without server management
- Delivers data near real-time

📌 Destinations:
- S3 for storage
- Redshift for analytics
- OpenSearch for search and dashboards

📌 Extra:
- Lambda can transform data before delivery
- Buffering controls delivery timing
- Supports compression and backup

☁️ **SQS vs SNS vs Kinesis**

📌 SQS:
- Pull-based system where consumers read messages
- Used for asynchronous background processing

📌 SNS:
- Push-based system delivering messages instantly
- Used for sending to multiple subscribers

📌 Kinesis:
- Real-time streaming platform with replay capability
- Used for analytics and continuous data flow

☁️ **Amazon MQ**

📌 Definition:
Amazon MQ is a managed message broker service used for applications that rely on traditional messaging protocols

📌 Key Points:
- Supports protocols like AMQP, MQTT, STOMP
- Useful for migrating legacy systems
- Runs on managed servers
- Supports both queues and topics

☁️ **Containers on AWS**

📌 What is Docker:
Docker is a software development platform used to build, package, and deploy applications using containers
- Applications are packaged into containers that run on any operating system
- Applications behave the same regardless of environment (no compatibility issues)
- Works with any language, any OS, and any technology
- Reduces setup effort and makes deployment easier

📌 Use Cases:
- Microservices architecture for scalable apps
- Lift-and-shift applications from on-prem to cloud
- Consistent environments across development and production

📌 Where Docker Images are Stored:
Docker images are stored in repositories
- Docker Hub → public repository with ready-to-use images
- Amazon ECR → private and public AWS repository integrated with services

📌 Docker vs Virtual Machines:
- Docker → shares host OS, lightweight, fast startup
- Virtual Machines → full OS per VM, heavier, slower

👉 Key Difference:
Docker uses containerization, while VM uses full virtualization

---

☁️ **Container Management on AWS**

📌 AWS Services:
- Amazon ECS → AWS native container platform
- Amazon EKS → managed Kubernetes service
- AWS Fargate → serverless container platform
- Amazon ECR → store container images

📌 Amazon ECS (Elastic Container Service):
Amazon ECS is a fully managed container orchestration service that allows you to run Docker containers on AWS

📌 Key Concepts:
- ECS Cluster → group of compute resources
- ECS Task → running container
- ECS Service → maintains and scales tasks

📌 Launch Types:
- EC2 Launch Type → you manage EC2 instances, ECS agent runs on each instance, more control
- Fargate Launch Type → no EC2 management, fully serverless, just define CPU and RAM

📌 IAM Roles in ECS:
- EC2 Instance Role → used by ECS agent to pull images, send logs, call AWS APIs
- Task Role → gives permissions to containers, defined in task

📌 Load Balancer Integration:
- Application Load Balancer → most common

- Network Load Balancer → high performance use cases
- Classic Load Balancer → not recommended

📌 Data Volumes (EFS):
- EFS can be mounted to ECS tasks
- Works with both EC2 and Fargate
- Shared storage across multiple AZs

👉 Important:
S3 cannot be mounted as a file system

📌 ECS Auto Scaling:
- Automatically adjusts number of tasks
- Based on CPU, memory, or ALB request count

📌 Types of Scaling:
- Target tracking → maintain specific metric value
- Step scaling → based on alarms
- Scheduled scaling → based on time

📌 EC2 Auto Scaling with ECS:
- Adds EC2 instances when capacity is low
- Uses Auto Scaling Groups
- Capacity Provider helps manage scaling

📌 Common Use Cases:
- Run tasks using EventBridge
- Process messages from SQS
- Batch processing jobs

☁️ **Amazon ECR (Elastic Container Registry)**

📌 Definition:
Amazon ECR is a container image registry used to store and manage Docker images

📌 Features:
- Supports private and public repositories
- Integrated with ECS and EKS
- Uses IAM for access control
- Supports image scanning, tagging, and lifecycle policies

☁️ **Amazon EKS (Elastic Kubernetes Service)**

📌 Definition:
Amazon EKS is a managed Kubernetes service that allows you to run Kubernetes clusters on AWS

📌 Key Points:
- Kubernetes is an open-source system for container orchestration
- Alternative to ECS with different APIs
- Works across multiple cloud providers

📌 Deployment Options:
- EC2 → manage worker nodes
- Fargate → serverless containers

📌 Use Case:
Used when organization already uses Kubernetes

📌 Node Types:
- Managed Nodes → AWS manages EC2
- Self-managed Nodes → you manage EC2
- Fargate → no nodes required

📌 Storage:
- Uses CSI drivers
- Supports EBS, EFS, and FSx

☁️ **AWS App Runner**

📌 Definition:
AWS App Runner is a fully managed service used to deploy web applications and APIs at scale

📌 Features:
- No infrastructure management required
- Automatic scaling and load balancing
- High availability and secure by default

- Supports VPC access

📌 Input:

- Source code or Docker container image

📌 Use Cases:

- Web applications
- APIs
- Microservices

☁️ **AWS App2Container (A2C)**

📌 Definition:

AWS App2Container is a tool used to migrate existing applications into Docker containers

📌 Features:

- Converts applications into containers without code changes
- Generates CloudFormation templates
- Supports CI/CD pipeline creation

📌 Process:

- Discover application and dependencies
- Extract and containerize
- Generate deployment configuration
- Deploy to ECS, EKS, or App Runner

☁️ **Serverless Overview**

📌 What is Serverless:

Serverless is a cloud computing model where developers do not need to manage servers, they only deploy code and functions

- No server provisioning or maintenance required, AWS manages infrastructure
- Initially serverless = Function as a Service (FaaS)
- Now includes databases, storage, messaging, APIs
- Automatically scales based on demand

👉 Important:

Serverless does NOT mean no servers, it means you don't manage them

📌 Serverless in AWS:

- AWS Lambda → run code
- DynamoDB → NoSQL database
- API Gateway → create APIs
- S3 → storage
- SNS / SQS → messaging
- Kinesis → streaming
- Cognito → authentication
- Aurora Serverless → database
- Step Functions → workflow
- Fargate → serverless containers

📌 EC2 vs Lambda:

- EC2 → servers run continuously and need manual scaling
- Lambda → runs on-demand, auto scaling, event-driven

👉 Lambda is event-based, EC2 is infrastructure-based

☁️ **AWS Lambda**

📌 Definition:

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing servers

📌 Key Features:

- Runs only when triggered by events
- Automatically scales with traffic
- Supports multiple programming languages
- Integrated with many AWS services
- Monitoring through CloudWatch

📌 Supported Languages:

- Node.js, Python, Java
- C# (.NET), Ruby
- Custom runtimes supported

📌 Deployment Options:
- ZIP file upload
- Container image deployment

📌 Integrations (Triggers):
- API Gateway, S3, DynamoDB
- SNS, SQS, Kinesis
- EventBridge, CloudWatch
- CloudFront

📌 Use Cases:
- File upload processing (S3 trigger)
- Scheduled jobs (EventBridge cron)
- Real-time data processing

📌 Pricing:
- Pay per request and execution duration
- First 1M requests free
- 400,000 GB-seconds free

📌 Limits:
- Max execution → 15 minutes
- Memory → 128 MB to 10 GB
- Deployment package → 50 MB (zip)
- Default concurrency → 1000

📌 Lambda SnapStart:
- Reduces cold start time (mainly Java)
- Uses pre-initialized execution state

☁️ **Lambda Edge & CloudFront Functions**

📌 Edge Computing Concept:
Run code closer to users to reduce latency

📌 CloudFront Functions:
- Lightweight, very fast execution
- Used for simple logic (headers, redirects)

📌 Lambda@Edge:
- More powerful, supports complex processing
- Can modify request/response

📌 Use Cases:
- Authentication and security
- URL rewrites and redirects
- Content customization
- A/B testing

👉 Difference:
- Functions → fast and simple
- Lambda@Edge → flexible and powerful

☁️ **Lambda in VPC & RDS Integration**

📌 Default Behavior:
- Lambda runs outside VPC
- Cannot access private resources

📌 To Connect VPC:
- Attach Lambda to VPC
- Configure subnets and security groups
- AWS creates ENI

📌 Lambda with RDS Proxy:
- Manages database connections efficiently
- Prevents overload on database
- Improves performance and scalability

📌 RDS + Lambda:
- RDS can trigger Lambda on events
- Used for automation and data processing

☁️ **DynamoDB**
📌 Definition:
DynamoDB is a fully managed NoSQL database with high performance, scalability, and low latency
📌 Basics:
- Data stored in tables
- Items = rows, attributes = columns
- Primary key required
- Max item size → 400 KB

📌 Data Types:
- Scalar → String, Number, Boolean
- Document → List, Map
- Set → String Set, Number Set

📌 Key Features:
- Multi-AZ replication
- Auto scaling
- Single-digit millisecond latency
- No maintenance required

📌 Capacity Modes:
- Provisioned → define RCU/WCU
- On-Demand → auto scaling

📌 DAX:
- In-memory cache
- Microsecond latency

📌 Streams:
- Captures insert/update/delete
- Triggers Lambda

📌 Global Tables:
- Multi-region replication
- Active-active

📌 TTL:
- Automatically deletes expired data

📌 Backups:
- Point-in-time recovery (35 days)
- On-demand backups

📌 S3 Integration:
- Export/import data for analytics

☁️ **API Gateway**
📌 Definition:
API Gateway is a service to create, publish, and manage APIs at scale
📌 Features:
- Works with Lambda (serverless APIs)
- Supports REST and WebSocket APIs
- Handles authentication and authorization
- Supports throttling, caching, versioning

📌 Integrations:
- Lambda
- HTTP backend
- AWS services

📌 Endpoint Types:
- Edge-Optimized → global users
- Regional → same region
- Private → inside VPC only

📌 Security:
- IAM
- Cognito
- Custom authorizers
- HTTPS (ACM)

☁️ **Step Functions**
📌 Definition:
Step Functions is a service used to build serverless workflows by coordinating multiple AWS services
📌 Features:
- Sequential and parallel execution
- Error handling and retries
- Timeout control
- Visual workflow design

📌 Use Cases:
- Order processing systems
- Data pipelines
- Microservice orchestration

☁️ **Amazon Cognito**
📌 Definition:
Cognito is a service for authentication, authorization, and user management
📌 Types:
- User Pools → user sign-up and login
- Identity Pools → provide AWS credentials

📌 User Pools:
- Sign-up and login system
- MFA support
- Email/phone verification
- Social login support

📌 Identity Pools:
- Provide temporary AWS credentials
- Allow direct AWS service access
- Uses IAM roles

☁️ **Serverless Architectures**
📌 Mobile Application (MyTodoList Requirements):
We want to build a mobile application with the following requirements
- Expose functionality using REST API over HTTPS
- Fully serverless architecture (no server management)
- Users should directly interact with their own S3 folder
- Authentication handled using a managed service
- Users can write and read data (mostly read-heavy workload)
- Database should scale and support high read throughput

📌 REST API Architecture Flow:
- Mobile Client → sends HTTPS request
- API Gateway → receives REST request
- Lambda → executes business logic
- DynamoDB → stores and retrieves data
- Cognito → authenticates users

👉 Flow:
Client → API Gateway → Lambda → DynamoDB
Authentication handled by Cognito
📌 Giving Users Access to S3:
- User authenticates using Cognito
- Cognito provides temporary AWS credentials
- User directly uploads/downloads files from S3

👉 Benefit:
- No need to pass file through backend
- Faster and cost-efficient

📌 High Read Throughput Optimization:
- DynamoDB used as main database
- DAX (DynamoDB Accelerator) used as caching layer

👉 Why:
- Application is read-heavy

- DAX improves performance (microsecond latency)

📌 API Gateway Caching:
- Cache API responses at API Gateway level
- Reduces calls to Lambda and DynamoDB

👉 Benefit:
- Faster responses
- Lower cost

📌 Key Learnings:
- API Gateway + Lambda + DynamoDB → serverless backend
- Cognito → authentication + temporary credentials
- Direct S3 access → scalable and efficient
- DAX → improves read performance
- API Gateway caching → reduces load

☁️ **Serverless Hosted Website (MyBlog)**

📌 Requirements:
- Website should scale globally
- Content is mostly read (rare writes)
- Mix of static and dynamic content
- Caching should be used wherever possible
- Send email when user subscribes
- Generate thumbnail when image uploaded

📌 Serving Static Content:
- Store static files in S3
- Use CloudFront for global distribution

👉 Benefit:
- Low latency
- High performance

📌 Secure Static Content:
- Use OAC (Origin Access Control)
- Only CloudFront can access S3

👉 Prevents direct public access

📌 Adding Serverless REST API:
- API Gateway → expose API
- Lambda → process logic
- DynamoDB → database
- DAX → caching layer

👉 Fully serverless backend

📌 Global Data Access:
- Use DynamoDB Global Tables
- Data replicated across regions

👉 Benefit:
- Low latency globally
- High availability

📌 User Welcome Email Flow:
- DynamoDB Streams detect changes
- Trigger Lambda
- Lambda uses SES (Simple Email Service) to send email

👉 Event-driven architecture

📌 Thumbnail Generation Flow:
- User uploads image to S3
- S3 triggers Lambda
- Lambda creates thumbnail
- Store thumbnail in S3

👉 Optional:
- Use SQS / SNS for decoupling

📌 Summary:
- S3 + CloudFront → static content
- API Gateway + Lambda → backend

- DynamoDB + DAX → database + caching
- Cognito → authentication
- SES → email service
- S3 events → trigger workflows

☁️ **Microservices Architecture**
📌 Concept:
Application is divided into multiple small independent services
- Each service has its own logic
- Services communicate via APIs

📌 Communication Types:
- Synchronous → API Gateway, Load Balancer
- Asynchronous → SQS, SNS, Kinesis, Lambda triggers

📌 Microservices Environment:
- Route 53 → DNS routing
- API Gateway / Load Balancer → entry point
- ECS / EC2 / Lambda → compute
- DynamoDB / RDS → database
- ElastiCache → caching

📌 Challenges:
- Managing many services
- Deployment complexity
- Resource utilization issues
- Version management

📌 Serverless Advantage:
- Auto scaling
- Pay per usage
- Easy to replicate environments
- API Gateway can generate SDK

☁️ **Cost Optimization using CloudFront**
📌 Problem:
- Application running on EC2
- Software updates downloaded by many users
- High CPU usage and network cost

📌 Solution:
- Add CloudFront in front of application
- Cache static files at edge locations

📌 Benefits:
- No change required in existing architecture
- Static files served from edge (faster)
- Reduced load on EC2 instances
- Lower cost (compute + bandwidth)
- Improved availability

☁️ **Databases in AWS**
📌 Choosing the Right Database:
AWS provides many database options, and selection depends on your use case
- Workload type → read-heavy, write-heavy, or balanced
- Throughput → how many requests per second and scaling needs
- Data size → how much data and how fast it will grow
- Data durability → how important data safety is
- Latency → how fast response should be
- Data model → structured, semi-structured, or unstructured
- Query type → joins, analytics, search, etc.
- Schema → fixed schema or flexible
- Cost → license cost or cloud-native option

👉 Key Idea:
Choose database based on **use case, not popularity**

📌 Database Types in AWS:

- RDBMS (SQL / OLTP) → RDS, Aurora (supports joins, transactions)
- NoSQL → DynamoDB, ElastiCache, DocumentDB, Neptune, Keyspaces
- Object Store → S3, Glacier (store large files)
- Data Warehouse (OLAP) → Redshift, Athena, EMR
- Search → OpenSearch (full-text search)
- Graph → Neptune (relationships data)
- Ledger → QLDB (immutable records)
- Time Series → Timestream (time-based data)

☁️ **Amazon RDS**

📌 Definition:

Amazon RDS is a managed relational database service that supports SQL databases like MySQL, PostgreSQL, Oracle, SQL Server, etc.

📌 Key Features:

- Managed database (no manual setup)
- Supports multiple database engines
- Storage auto-scaling
- Read Replicas for scaling reads
- Multi-AZ for high availability
- Automated backups (up to 35 days)
- Manual snapshots for long-term backup
- Security using IAM, KMS, SSL
- Maintenance handled by AWS

📌 Use Case:

- Relational data
- SQL queries
- Transactions (banking, apps)

☁️ **Amazon Aurora**

📌 Definition:

Amazon Aurora is a cloud-native relational database compatible with MySQL and PostgreSQL with better performance and scalability

📌 Key Features:

- Storage and compute are separated
- Data stored in 6 replicas across 3 AZ
- Self-healing and auto-scaling storage
- Supports multiple read replicas
- Custom endpoints for read/write

📌 Advanced Features:

- Aurora Serverless → auto scaling, no capacity planning
- Aurora Global → multi-region replication (<1 sec delay)
- Aurora ML → integrate with ML services
- Database Cloning → fast copy of database

📌 Use Case:

- Same as RDS but better performance and scalability

☁️ **Amazon ElastiCache**

📌 Definition:

Amazon ElastiCache is a managed in-memory database service using Redis or Memcached

📌 Key Features:

- Sub-millisecond latency
- In-memory storage (very fast)
- Supports clustering and replication
- Multi-AZ support
- Backup and restore support
- Security using IAM, KMS

📌 Important:

- Requires application code changes

📌 Use Case:
- Cache database queries
- Session storage
- Frequently accessed data

☁️ **Amazon DynamoDB**
📌 Definition:
DynamoDB is a fully managed serverless NoSQL database with high performance and scalability
📌 Key Features:
- Serverless and auto scaling
- Multi-AZ by default
- Millisecond latency
- No schema required
- Transaction support

📌 Capacity Modes:
- Provisioned → fixed capacity with auto scaling
- On-demand → automatic scaling

📌 Advanced Features:
- DAX → in-memory cache for faster reads
- Streams → capture data changes
- Global Tables → multi-region replication
- TTL → auto delete old data
- Backup → up to 35 days

📌 Use Case:
- Serverless applications
- Key-value storage
- High scale applications

☁️ **Amazon S3 (as Database)**
📌 Definition:
Amazon S3 is an object storage service used as a key-value store for large files
📌 Key Features:
- Unlimited storage
- Max object size → 5 TB
- Multiple storage classes (Standard, IA, Glacier)
- Versioning and replication
- Encryption and security policies

📌 Use Case:
- Store images, videos, backups

☁️ **Amazon Keyspaces (Cassandra)**
📌 Definition:
Amazon Keyspaces is a managed Apache Cassandra-compatible NoSQL database
📌 Key Features:
- Serverless and scalable
- Multi-AZ replication
- Uses Cassandra Query Language (CQL)
- Millisecond latency

📌 Use Case:
- IoT data
- Time-series workloads

☁️ **Data & Analytics**
📌 **Amazon Athena**
📌 Definition:
Amazon Athena is a serverless query service used to analyze data stored in Amazon S3 using standard SQL
- No infrastructure setup required
- Uses SQL (built on Presto engine)
- Supports formats → CSV, JSON, ORC, Avro, Parquet
- Pricing based on data scanned ($5 per TB)

- Works with QuickSight for dashboards

☁️ **Amazon Redshift**
📌 Definition:
Amazon Redshift is a data warehouse service used for analytics (OLAP), not for transactional workloads
- Based on PostgreSQL but optimized for analytics
- Uses columnar storage and parallel processing
- Can scale to petabytes of data
- Supports SQL queries
- Integrates with BI tools (QuickSight, Tableau)

☁️ **Amazon OpenSearch**
📌 Definition:
Amazon OpenSearch is a search and analytics service used for full-text search and log analysis
- Successor of Elasticsearch
- Can search any field (even partial matches)
- Used along with databases (not replacement)

☁️ **Amazon EMR (Elastic MapReduce)**
📌 Definition:
Amazon EMR is a service used to run big data processing frameworks like Hadoop and Spark
- Creates cluster of EC2 instances
- Used for large-scale data processing

📌 Features:
- Supports Spark, Hadoop, HBase, Flink
- Auto scaling
- Integrated with Spot instances
- Fully managed cluster setup

📌 Node Types:
- Master Node → manages cluster
- Core Node → processes and stores data
- Task Node → only processes data

📌 Purchasing Options:
- On-Demand → reliable
- Reserved → cost saving
- Spot → cheaper but can terminate

📌 Use Cases:
- Big data processing
- Machine learning
- Data pipelines

☁️ **Amazon QuickSight**
📌 Definition:
QuickSight is a serverless business intelligence (BI) service used to create dashboards and visualizations
📌 Features:
- Interactive dashboards
- Auto scaling
- Pay per session
- Supports embedding

☁️ **AWS Glue**
📌 Definition:
AWS Glue is a managed ETL (Extract, Transform, Load) service used to prepare data for analytics
📌 Features:
- Fully serverless
- Converts and transforms data

☁️ **AWS Lake Formation**
📌 Definition:
Lake Formation is a service used to build and manage a data lake

📌 Features:
- Central storage of structured and unstructured data
- Automates data collection and cleaning
- Built on top of AWS Glue
- Fine-grained access control (row/column level)

📌 Use Case:
- Central analytics platform
- Combine data from multiple sources

☁️ **Kinesis Data Analytics**

📌 Definition:
Service to process real-time streaming data using SQL or Apache Flink

📌 SQL Application:
- Run SQL queries on Kinesis streams
- Fully managed and auto scaling
- Output → S3, Redshift, Kinesis

📌 Use Cases:
- Real-time dashboards
- Time-series analytics
- Live metrics

📌 Flink Application:
- Use Java/Scala/SQL
- Advanced stream processing
- Supports complex transformations

☁️ **Amazon MSK (Managed Kafka)**

📌 Definition:
Amazon MSK is a fully managed Apache Kafka service used for real-time streaming

📌 Features:
- Managed Kafka clusters
- Runs inside VPC
- Multi-AZ for high availability
- Stores data on EBS
- Serverless option available

📌 Kafka Concepts:
- Producers → send data
- Brokers → store data
- Consumers → read data
- Topics → data streams

📌 Kinesis vs MSK:
- Kinesis → AWS native, shard-based
- MSK → Kafka-based, partition-based

☁️ **Big Data Ingestion Pipeline**

📌 Requirements:
- Collect real-time data
- Transform data
- Store data in S3
- Query using SQL
- Build dashboards

📌 Architecture Flow:
- IoT Devices → send data
- Kinesis Data Streams → collect data
- Kinesis Firehose → deliver to S3
- Lambda → transform data
- S3 → store data

📌 Processing & Analytics:
- Athena → query data
- Redshift → data warehouse

- QuickSight → dashboards

📌 Optional Flow:
- S3 → SQS → Lambda → processing

📌 Key Learnings:
- Kinesis → real-time ingestion
- Firehose → delivery to S3
- Lambda → transformation
- Athena → serverless SQL
- QuickSight → visualization

☁️ **Monitoring (CloudWatch + EventBridge + Insights)**

📌 Amazon CloudWatch:

Amazon CloudWatch is a monitoring service used to collect metrics, logs, and events for AWS resources and applications

📌 CloudWatch Metrics:
- Metric → value to monitor (CPUUtilization, NetworkIn, etc.)
- Metrics are grouped into namespaces
- Dimensions → extra info like instanceId, environment
- Supports up to 30 dimensions per metric
- Each metric has timestamp-based data
- Can create dashboards for visualization
- Can create custom metrics (like RAM, app-specific data)

📌 Metric Streams:
- Stream metrics in near real-time continuously
- Send to Kinesis Data Firehose → then S3, Redshift, etc.
- Can integrate with external tools (Datadog, Splunk)
- Option to filter only required metrics

📌 CloudWatch Logs:
- Log Group → collection of logs (application level)
- Log Stream → logs from specific instance/container
- Retention can be set (1 day → 10 years or never expire)

📌 Logs Destinations:
- S3 (for storage and backup)
- Kinesis Streams / Firehose (for streaming)
- Lambda (for processing)
- OpenSearch (for search and analytics)

📌 Logs Sources:
- EC2 (via CloudWatch Agent)
- Lambda functions
- ECS containers
- API Gateway
- CloudTrail logs
- VPC Flow Logs
- Route53 DNS logs

📌 Logs Insights:
- Query and analyze logs using query language
- Example → find ERROR logs, count requests, filter IPs
- Works across multiple log groups
- Not real-time, used for analysis

📌 Logs Export vs Subscription:
- Export to S3 → delayed (can take hours)
- Subscription → near real-time streaming

📌 Subscription Targets:
- Kinesis Data Streams
- Kinesis Firehose
- Lambda

📌 EC2 Logs Setup:
- Logs are NOT sent by default
- Need CloudWatch Agent installed

- Can also work for on-prem servers

📌 Agents:
- Logs Agent → sends only logs
- Unified Agent → sends logs + system metrics (RAM, CPU, processes)

📌 CloudWatch Alarms:
- Trigger alerts based on metric thresholds
- Alarm States → OK, INSUFFICIENT_DATA, ALARM
- Supports high-resolution monitoring

📌 Alarm Actions:
- Stop / terminate / reboot EC2
- Trigger Auto Scaling
- Send notification using SNS

📌 Composite Alarms:
- Combine multiple alarms using AND / OR
- Helps reduce unnecessary alerts (noise)

📌 EC2 Instance Recovery:
- Automatically recover instance on failure
- Keeps same IP, metadata, and configuration

📌 EventBridge:
- Used for event-driven architecture
- Can schedule jobs (cron) or react to AWS events

📌 Event Targets:
- Lambda
- SQS / SNS
- Step Functions
- ECS Tasks

📌 EventBridge Features:
- Default bus (AWS events)
- Custom bus (application events)
- Partner bus (SaaS integrations)
- Can archive and replay events
- Schema registry helps understand event structure
- Supports cross-account access

📌 CloudWatch Insights (Advanced Monitoring):
- Container Insights → monitor ECS, EKS
- Lambda Insights → detailed Lambda metrics
- Contributor Insights → find top users/IPs
- Application Insights → detect application issues automatically

---

☁️ **Audit & Governance (CloudTrail)**

📌 Definition:
CloudTrail is a service used for governance, compliance, and auditing by recording all API activity in your AWS account

📌 Key Features:
- Enabled by default in AWS
- Tracks API calls from Console, CLI, SDK, and AWS services
- Logs can be stored in S3 or CloudWatch

📌 Event Types:
- Management Events → resource changes (default enabled)
- Data Events → S3 object access, Lambda execution
- Insights Events → detect unusual activity

📌 CloudTrail Insights:
- Detect anomalies like unusual API spikes
- Uses baseline behavior for comparison
- Sends alerts using EventBridge

📌 Retention:
- Default → 90 days
- Long-term → store logs in S3 and query using Athena

📌 Use Case:

- If resource deleted or modified → check CloudTrail

📌 Integration:

- Works with EventBridge to trigger alerts
- Example → DeleteTable API → trigger SNS alert

☁️ **Compliance & Configuration (AWS Config)**

📌 Definition:

AWS Config is a service used to track configuration changes and evaluate compliance of AWS resources over time

📌 Key Features:

- Records configuration history of resources
- Tracks changes over time
- Sends alerts via SNS
- Works per region (can aggregate across regions/accounts)
- Stores configuration data in S3

📌 Config Rules:

- Managed rules (predefined by AWS)
- Custom rules using Lambda

📌 Trigger Types:

- On configuration change
- On scheduled interval

👉 Important:

- Does NOT block actions (only monitors and reports)

📌 Resource View:

- View configuration history
- View compliance timeline
- View related CloudTrail events

📌 Auto Remediation:

- Automatically fix non-compliant resources
- Uses SSM Automation
- Can retry remediation

📌 Notifications:

- Use EventBridge or SNS
- Trigger Lambda / SQS for actions

☁️ **CloudWatch vs CloudTrail vs Config**

📌 CloudWatch:

- Used for monitoring (metrics, logs, performance)
- Provides alerts and dashboards

📌 CloudTrail:

- Used for auditing API activity
- Tracks who did what and when

📌 Config:

- Used for tracking configuration changes
- Ensures compliance of resources

📌 Example (Load Balancer):

- CloudWatch → monitor performance and traffic
- Config → track configuration changes and compliance
- CloudTrail → track who modified settings

☁️ **Advanced Identity in AWS**

📌 AWS Organizations:

AWS Organizations is a global service that allows you to manage multiple AWS accounts centrally

- Management Account → main account with full control
- Member Accounts → other accounts under organization
- One account can belong to only one organization
- Consolidated billing → single payment method for all accounts
- Cost benefits → shared discounts (EC2, S3, Reserved Instances, Savings Plans)
- Supports automation using APIs (create/manage accounts)

📌 Organizational Units (OU):

- Used to group accounts logically
- Example grouping:
  - By Business Unit → Sales, Finance
  - By Environment → Dev, Test, Prod
  - By Project → Project-based accounts

👉 Helps in applying policies and managing access easily

📌 Best Practices:
- Use multiple accounts instead of one large account
- Use tagging for cost tracking
- Enable CloudTrail for all accounts (central logging S3)
- Send CloudWatch logs to centralized logging account
- Use cross-account roles for admin access

📌 Service Control Policies (SCP):
SCPs are policies used to control permissions at organization level
- Applied to OU or accounts
- Restrict what users and roles can do
- Do NOT grant permissions (only limit them)
- Do NOT apply to management account

👉 Important Behavior:
- By default → nothing is allowed
- Need explicit allow from root → OU → account
- Explicit DENY always overrides allow

📌 Example Understanding:
- If OU denies S3 → all accounts under it cannot use S3
- Even if IAM allows → SCP deny will block it

📌 SCP Strategies:
- Allowlist → allow only specific services
- Blocklist → allow everything except restricted services

📌 IAM Conditions:
Conditions add extra security rules to IAM policies
- aws:SourceIp → restrict access based on IP
- aws:RequestedRegion → restrict AWS region usage
- ec2:ResourceTag → restrict based on tags
- aws:MultiFactorAuthPresent → enforce MFA

👉 Used for fine-grained control

📌 IAM for S3 (Important):
- s3:ListBucket → applies to bucket level
- s3:GetObject / PutObject → applies to object level

👉 Bucket vs Object permissions are different

📌 Resource Policies & aws:PrincipalOrgID:
- Used in resource-based policies (like S3 bucket policy)
- Restricts access only to accounts within organization

👉 Example:
- Only org accounts can access S3 bucket
- External users are blocked

📌 IAM Roles vs Resource-Based Policies:

📌 Cross Account Access:
- Option 1 → Use IAM Role (assume role)
- Option 2 → Use Resource Policy (direct access)

📌 Key Difference:
- IAM Role → user gives up original permissions and uses role permissions
- Resource Policy → user keeps original permissions

📌 Example:
- Account A user → access S3 bucket in Account B

- Can use role OR bucket policy

📌 EventBridge Security:
- EventBridge rules need permission to access targets
- Two ways:
  - Resource-based policy → Lambda, SNS, SQS
  - IAM Role → EC2, ECS, SSM

📌 IAM Permission Boundaries:
IAM Permission Boundaries define the **maximum permissions** a user or role can have
- Works only for users and roles (not groups)
- Acts as upper limit

👉 Final permission = IAM Policy + Boundary
📌 Use Cases:
- Allow developers to create users but restrict permissions
- Prevent privilege escalation
- Delegate admin tasks safely

📌 IAM Policy Evaluation Logic:
AWS checks multiple policies together:
- IAM policies
- SCPs
- Resource policies
- Permission boundaries

👉 Rule:
- Explicit DENY always wins
- Otherwise → must have explicit ALLOW

📌 AWS IAM Identity Center (SSO):
IAM Identity Center provides single sign-on (SSO) for AWS accounts and applications
- One login for multiple AWS accounts
- Supports business apps (Salesforce, Microsoft 365, etc.)
- Works with SAML 2.0 apps
- Supports EC2 Windows login

📌 Identity Sources:
- Built-in identity store
- Active Directory
- Third-party providers (Okta, OneLogin)

📌 IAM Identity Center Concepts
- Permission Sets → group of IAM policies
- Assigned to users/groups
- Used across multiple AWS accounts

📌 Features:
- Multi-account access management
- Centralized user management
- Supports Attribute-Based Access Control (ABAC)

👉 Example:
- Developers group → ReadOnlyAccess in Dev
- Admin group → FullAccess in Prod

📌 Microsoft Active Directory (AD):
- Central directory of users, computers, resources
- Used for authentication and authorization
- Organized in domains → trees → forests

📌 AWS Managed Microsoft AD:
- Fully managed AD in AWS
- Can connect with on-prem AD (trust relationship)

📌 AD Connector:
- Proxy to on-prem AD
- No user stored in AWS

📌 Simple AD:
- Basic AD-compatible directory
- Cannot connect with on-prem AD

📌 IAM Identity Center + AD Integration:
- Can connect with AWS Managed AD
- Can connect with on-prem AD using AD Connector
- Supports trust relationships

👉 Used for enterprise login systems

☁️ **AWS Control Tower**

📌 Definition:
AWS Control Tower is a service that helps set up and manage a secure multi-account AWS environment using best practices

📌 Features:
- Uses AWS Organizations in background
- Automates account creation
- Applies governance rules
- Provides compliance dashboard

📌 Benefits:
- Quick setup of environment
- Centralized management
- Automated policy enforcement
- Continuous compliance monitoring

📌 Guardrails (Important):
Guardrails are rules applied to accounts for governance

📌 Types:
- Preventive Guardrails → use SCP
  - Example → restrict regions
- Detective Guardrails → use AWS Config
  - Example → detect untagged resources

👉 Flow:
- Detect issue → trigger alert (SNS) → fix using Lambda

☁️ **AWS Security & Encryption**

📌 Why Encryption:
Encryption is used to protect data from unauthorized access and attacks

📌 Types of Encryption:
- Encryption in Transit (In-Flight):
  - Data is encrypted before sending and decrypted after receiving
  - Uses TLS/SSL (HTTPS)
  - Prevents MITM (Man-In-The-Middle) attacks
- Encryption at Rest (Server-Side):
  - Data is encrypted after reaching server
  - Stored in encrypted form using keys
  - Decrypted when accessed
- Client-Side Encryption:
  - Data is encrypted before sending to server
  - Server cannot decrypt data
  - Only client can decrypt

👉 Key Idea:
- In-flight → data moving
- At-rest → data stored
- Client-side → highest control (server can't see data)

📌 AWS KMS (Key Management Service):
AWS KMS is a service used to create and manage encryption keys

- Fully integrated with AWS services (S3, EBS, RDS, etc.)
- Integrated with IAM for access control
- Tracks usage using CloudTrail
- Used whenever encryption is enabled in AWS
- Never store secrets in plaintext

📌 KMS Key Types:

- Symmetric Keys:
  - Same key used for encrypt & decrypt
  - Most commonly used (AES-256)
  - Used by AWS services
- Asymmetric Keys:
  - Public key (encrypt) + Private key (decrypt)
  - Used for external encryption use cases

📌 KMS Key Types (Ownership):

- AWS Owned Keys → default, free
- AWS Managed Keys → managed by AWS (aws/service-name)
- Customer Managed Keys → created by user ($1/month)
- Imported Keys → user provides keys

📌 Pricing:

- $1/month per key
- $0.03 per 10,000 API calls

📌 Key Rotation:

- AWS Managed → auto every 1 year
- Customer Managed → can enable auto rotation
- Imported Keys → manual rotation

📌 KMS Key Policies:

- Control access to KMS keys
- Similar to resource policies
- Required to use key

📌 Types:

- Default → full access to root account
- Custom → define users/roles access

👉 Used for cross-account key sharing

📌 KMS Multi-Region Keys:

- Same key available in multiple regions
- Same key ID and key material
- Can encrypt in one region and decrypt in another

👉 Use Cases:

- Global applications
- DynamoDB Global Tables
- Aurora Global DB

📌 Snapshot & AMI Encryption:

📌 Cross Region:

- Snapshot re-encrypted with new KMS key in target region

📌 Cross Account:

- Share snapshot + share KMS key
- Target account creates copy using its own key

📌 AMI Sharing:

- Share AMI + KMS key
- Target can re-encrypt using its own key

📌 S3 Encryption & Replication:

- SSE-S3 → replicated automatically
- SSE-KMS → need extra configuration
- Must define target KMS key
- Need IAM role permissions (decrypt source, encrypt target)

📌 SSM Parameter Store:
SSM Parameter Store is used to store configuration and secrets securely

- Supports plaintext and encrypted values
- Uses KMS for encryption
- Serverless and scalable
- Version tracking available
- Integrated with IAM

📌 Features:

- Store config (DB URL, API keys)
- Secure secrets management
- EventBridge notifications
- Works with CloudFormation

📌 Hierarchy Example:

- /app/dev/db-url
- /app/prod/db-password

📌 Parameter Store Tiers:

- Standard:
  - 10,000 params
  - 4 KB size
  - Free
- Advanced:
  - 100,000 params
  - 8 KB size
  - Supports policies
  - Paid

📌 Parameter Policies (Advanced):

- Expiration → auto delete
- ExpirationNotification → notify before expiry
- NoChangeNotification → alert if not updated

📌 AWS Secrets Manager:
Secrets Manager is used to store sensitive data like passwords

- Automatically rotates secrets
- Uses Lambda for rotation
- Integrated with RDS
- Encrypted using KMS

👉 Difference from Parameter Store:

- Secrets Manager → rotation support
- Parameter Store → simpler & cheaper

📌 Multi-Region Secrets:

- Replicate secrets across regions
- Used for disaster recovery

📌 AWS Certificate Manager (ACM):
ACM is used to create and manage SSL/TLS certificates

- Provides HTTPS encryption
- Supports public and private certificates
- Free for public certificates
- Automatic renewal

📌 Integration:

- Load Balancer
- CloudFront

- API Gateway

👉 Important:
- Cannot use ACM cert directly on EC2

📌 ACM Certificate Process:
- Add domain (example.com, *.example.com)
- Validate via DNS (recommended) or Email
- Certificate issued in few hours
- Auto-renew before expiry

📌 Import Certificates:
- Can import external cert
- No auto-renewal
- Expiry tracked via EventBridge

📌 API Gateway Security:

📌 Endpoint Types:
- Edge-Optimized → global (via CloudFront)
- Regional → same region
- Private → inside VPC only

📌 AWS WAF (Web Application Firewall):

WAF protects applications from common web attacks
- Works at Layer 7 (HTTP level)
- Used with ALB, API Gateway, CloudFront

📌 Features:
- IP filtering
- Block SQL Injection, XSS
- Rate limiting (DDoS protection)
- Geo-blocking

📌 AWS Shield:
- Protects against DDoS attacks

📌 Types:
- Shield Standard → free, basic protection
- Shield Advanced → paid, advanced protection

📌 Features:
- Protects EC2, ELB, CloudFront, Route53
- 24/7 support team
- Cost protection during attack

📌 AWS Firewall Manager:
- Manage security rules across accounts (Organizations)

📌 Supports:
- WAF rules
- Shield Advanced
- Security Groups
- Network Firewall

👉 Automatically applies rules to new resources

📌 WAF vs Shield vs Firewall Manager:
- WAF → protect web apps (Layer 7)
- Shield → DDoS protection
- Firewall Manager → manage rules across accounts

📌 DDoS Protection Best Practices:
- Use CloudFront → edge protection
- Use Route53 → DNS protection
- Use Load Balancer → distribute traffic
- Use Auto Scaling → handle traffic spikes

- Use WAF → block malicious requests

📌 Amazon GuardDuty:
- Threat detection service using ML

📌 Monitors:
- CloudTrail logs
- VPC Flow Logs
- DNS logs

📌 Detects:
- Unusual API calls
- Suspicious traffic
- Crypto mining attacks

📌 Integration:
- EventBridge → alerts
- SNS / Lambda → actions

📌 Amazon Inspector:
- Security assessment service

📌 Works on:
- EC2
- Container images (ECR)
- Lambda

📌 Checks:
- Vulnerabilities (CVE)
- Network exposure

📌 AWS Security Hub:
- Central dashboard for security findings

📌 Integrates with:
- GuardDuty
- Inspector
- Macie

📌 AWS Macie:
- Data security service

📌 Features:
- Detect sensitive data (PII)
- Scan S3 buckets
- Uses ML + pattern matching

🚀 **Amazon VPC (Virtual Private Cloud)**

🧠 **One Line**

: VPC = a private network in AWS where you run your resources

🔑 **Core Components**
- VPC → network boundary
- Subnet → network division (per Availability Zone)
- Route Table → defines traffic paths
- Internet Gateway (IGW) → enables internet access
- NAT Gateway → allows private subnet to access internet
- Security Group → instance-level firewall
- NACL → subnet-level firewall

🌐 **Subnets**
- Public Subnet → has route to IGW, internet access
- Private Subnet → no direct internet, uses NAT Gateway

🌍 **Internet Access Flow**
- Internet → IGW → Public Subnet → EC2
- Private Subnet → NAT Gateway → IGW → Internet

🔐 **Security**

Security Group

- Instance level
- Stateful
- Allow only

NACL

- Subnet level
- Stateless
- Allow and Deny

🔗 **VPC Peering**

- Connect two VPCs privately
- No overlapping CIDR
- Not transitive

🔌 **VPC Endpoints**

- Access AWS services without internet
- Gateway Endpoint → S3, DynamoDB (free)
- Interface Endpoint → other services (paid)

📊 **VPC Flow Logs**

- Capture network traffic
- Used for debugging
- Send to CloudWatch or S3

🌉 **Connectivity**

- Site-to-Site VPN → on-prem to AWS over internet
- Direct Connect → dedicated private connection

🔥 **Transit Gateway**

- Connect multiple VPCs
- Hub-and-spoke model

🎯 **CIDR Basics**

- Defines IP range
- 10.0.0.0/16 → large network
- 192.168.0.0/24 → small network
- 0.0.0.0/0 → all IPs

⚠️ **Important Notes**

- Each subnet reserves 5 IP addresses
- VPC CIDR must not overlap
- Default VPC has internet access

🧠 **Architecture Example**

- Internet → IGW → Public Subnet → ALB
  ↓
- Private Subnet → EC2 → RDS

🎯 **Final Summary**

- VPC = private network
- Public = internet access
- Private = secure backend
- NAT = private to internet
- Security Group and NACL = protection

🚀 **Disaster Recovery (DR)**

🧠 **One Line**

👉 DR = minimize data loss (RPO) + downtime (RTO)

🔑 **Key Concepts**

- **RPO (Recovery Point Objective)** → Kitna data loss acceptable hai
- **RTO (Recovery Time Objective)** → System kitni jaldi recover hona chahiye

🔥 **DR Strategies**

1. Backup & Restore

- Data S3 / snapshots me backup
- Disaster pe restore
  👉 Cheapest, but slow

2. Pilot Light

- Core system (DB) running

- App servers off
  👉 Faster than backup

3. Warm Standby

- Full system running at small scale
- Disaster pe scale up
  👉 Balanced cost + speed

4. Hot Site (Multi-Site)

- Full system active in multiple regions
  👉 Instant failover, but expensive

📊 **DR Comparison**

Strategy	Cost	Recovery Speed	Use Case
Backup	Low	Slow	Low priority apps
Pilot Light	Medium	Medium	Critical DB apps
Warm Standby	Medium	Fast	Production apps
Hot Site	High	Instant	Zero downtime

🌍 **Failover Flow**

Primary Region → Route53 → Failover → Secondary Region

🚀 **Migration**

🔄 **DMS (Database Migration Service)**

👉 Database migration tool

- Live migration (no downtime)
- Uses CDC (Change Data Capture)
- Homogeneous (same DB)
- Heterogeneous (different DB)

Source DB → DMS → Target DB

⚙️ **MGN (Application Migration Service)**

👉 Lift & Shift migration

- Move servers (OS + apps + data)
- Minimal downtime

📦 **Snowball**

👉 Large data transfer (TB / PB scale)

- Physical device
- Faster than internet

🚀 **Architecture Add-ons**

🧱 **CloudFormation**

👉 Infrastructure as Code

- Create full environment automatically
- Re-create infra during disaster

⚡ **Event-Driven Architecture**

- SQS → queue
- SNS → pub/sub
- Lambda → processing